

Back-end em python



Prof^a. Ana Clara

SOFTEX
PERNAMBUCO

 **Softex**

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO

GOVERNO FEDERAL
BRASIL
UNIÃO E RECONSTRUÇÃO



Aula 22 - Modelo Lógico Relacional, Chaves e Normalização



Objetivos da Aula

- Compreender o modelo lógico relacional derivado do modelo ER
- Entender o papel das chaves primária e estrangeira
- Aprender os princípios da normalização de dados
- Realizar a modelagem lógica de um sistema de exemplo



Aula 22 - Modelo Lógico Relacional, Chaves e Normalização

Do modelo ER ao modelo relacional

- Tradução do diagrama ER para tabelas relacionais
- Cada entidade vira uma tabela
- Cada relacionamento vira uma tabela ou chave estrangeira





Aula 22 - Modelo Lógico Relacional, Chaves e Normalização



O que é Modelo Lógico Relacional?

- Representação das entidades e relacionamentos em **tabelas**
- Cada entidade → tabela
- Cada atributo → coluna
- Relacionamentos → chaves estrangeiras



Aula 22 - Modelo Lógico Relacional, Chaves e Normalização

Tabela relacional

- Conjunto de tuplas (linhas)
- Cada tupla representa um registro
- Linhas = registro (tuplas)
- Colunas = atributos (campos)
- Ordem das linhas e colunas não importa logicamente





Aula 22 - Modelo Lógico Relacional, Chaves e Normalização



Exemplo em SQL:

sql

```
CREATE TABLE Aluno (  
    id_aluno INT PRIMARY KEY,  
    nome VARCHAR(100),  
    idade INT  
);
```



Aula 22 - Modelo Lógico Relacional e Normalização



Chave Primária (PK)

- Identifica unicamente cada linha da tabela
- Pode ser natural (ex: CPF) ou artificial (ex: id)
- Não pode conter valores nulos ou repetidos

```
sql

CREATE TABLE Curso (
    id_curso INT PRIMARY KEY,
    nome VARCHAR(100)
);
```



Aula 22 - Modelo Lógico Relacional e Normalização



Chave Estrangeira (FK)

- Faz referência a uma PK em outra tabela
- Estabelece vínculos entre tabelas
- Garante integridade referencial
- Usada para implementar relacionamentos

sql

```
CREATE TABLE Matricula (  
    id_matricula INT PRIMARY KEY,  
    id_aluno INT,  
    id_curso INT,  
    FOREIGN KEY (id_aluno) REFERENCES Aluno(id_aluno),  
    FOREIGN KEY (id_curso) REFERENCES Curso(id_curso)  
);
```




Aula 22 - Modelo Lógico Relacional e Normalização



Chave Candidata

- Atributo que **poderia ser chave primária**
- Exemplo: em Cliente, tanto CPF quanto Email podem identificar unicamente



Aula 22 - Modelo Lógico Relacional e Normalização

Exemplo prático com PK e FK

- Tabela `cliente` (`id_cliente` PK, `nome`)
- Tabela `pedido` (`id_pedido` PK, `id_cliente` FK)
- Cada pedido está vinculado a um cliente





Aula 22 - Modelo Lógico Relacional , Chaves e Normalização

Chave Alternativa

- Quando existem várias candidatas, a escolhida como PK é a principal, e as demais são **chaves alternativas**.





Aula 22 - Modelo Lógico Relacional , Chaves e Normalização

Cardinalidade no Modelo Lógico

- 1:N → FK simples
- N:M → tabela intermediária
- Exemplo: Alunos x Disciplinas





Aula 22 - Modelo Lógico Relacional e Normalização

Cardinalidade no Modelo Lógico

- 1:N → FK simples
- N:M → tabela intermediária
- Exemplo: Alunos x Disciplinas





Aula 22 - Modelo Lógico Relacional e Normalização

Do ER para Tabelas

Exemplo ER → **Aluno (1)**
curso (N) Curso

```
sql

CREATE TABLE Aluno (
  id_aluno INT PRIMARY KEY,
  nome VARCHAR(100)
);

CREATE TABLE Curso (
  id_curso INT PRIMARY KEY,
  nome VARCHAR(100)
);

CREATE TABLE Matricula (
  id_aluno INT,
  id_curso INT,
  PRIMARY KEY (id_aluno, id_curso),
  FOREIGN KEY (id_aluno) REFERENCES Aluno(id_aluno),
  FOREIGN KEY (id_curso) REFERENCES Curso(id_curso)
);
```





Aula 22- Modelo Lógico Relacional e Normalização



O que é normalização?

Processo de organização dos dados para:

- Eliminar redundâncias
- Melhorar integridade
- Reduzir anomalias de atualização

Etapas:

- . 1FN → elimina grupos repetidos
- . 2FN → remove dependência parcial
- . 3FN → remove dependência transitiva



Aula 22- Modelo Lógico Relacional e Normalização



Primeira Forma Normal (1FN)

- Cada campo deve conter apenas um valor atômico.
- Exemplo (NÃO normalizado):

Cliente	Telefones
Ana	9999-1111, 8888-2222

✓ Correto (1FN):

Cliente	Telefone
Ana	9999-1111
Ana	8888-2222



Aula 22- Modelo Lógico Relacional e Normalização

Primeira forma normal (1FN)

- Elimina atributos multivalorados ou repetidos
- Cada campo deve conter **um único valor atômico**
- Exemplo incorreto: `telefones = ["9999", "8888"]`





Aula 22- Modelo Lógico Relacional e Normalização

Segunda Forma Normal (2FN)

- Deve estar em 1FN
- Eliminar **dependência parcial** (quando atributo depende só de parte da chave composta).





Aula 22- Modelo Lógico Relacional e Normalização

Exemplo 2FN

Tabela original:

| AlunoID | CursoID | NomeCurso |

→ Problema: NomeCurso depende apenas de CursoID.

Solução: separar em duas tabelas (AlunoCurso e Curso).





Aula 22- Modelo Lógico Relacional e Normalização



Terceira Forma Normal (3FN)

- Deve estar em 2FN
- Eliminar **dependência transitiva** (atributo depende de outro que não é chave).



Aula 22- Modelo Lógico Relacional e Normalização



Exemplo 3FN

Tabela original:

| PedidoID | ClienteID | NomeCliente |

→ Problema: NomeCliente depende de ClienteID, não de PedidoID.

Solução: Criar tabela Cliente separada.



Aula 22- Modelo Lógico Relacional e Normalização

Benefícios da normalização

- Banco mais eficiente
- Menor uso de espaço, reduz redundância
- Facilidade de manutenção
- Dados consistentes





Aula 22- Modelo Lógico Relacional e Normalização

Exemplo Prático Completo

Sistema de Biblioteca:

- Tabelas: Livro, Autor, Usuario, Emprestimo
- Relacionamentos via FK
- Normalização aplicada até 3FN





Aula 22 - Modelo Lógico Relacional e Normalização



Exemplo prático de normalização (antes) Tabela única:

pedido(id_pedido, cliente_nome, cliente_email, produto_nome, valor)

Problemas: dados repetidos, difícil manutenção



Aula 22 - Modelo Lógico Relacional e Normalização

Exemplo normalizado (após) Tabelas:

- cliente(id_cliente, nome, email)
- produto(id_produto, nome, valor)
- pedido(id_pedido, id_cliente)
- item_pedido(id_pedido, id_produto)





Aula 22 - Modelo Lógico Relacional e Normalização

Regras para criar o modelo lógico

- Nome claro para tabelas e colunas
- Definir PKs e FKs desde o início
- Tabelas de junção para N:N
- Atributos normalizados (1 valor por campo)





Aula 22 - Modelo Lógico Relacional e Normalização

Atividade Prática 1

Transforme este ER em modelo lógico relacional:

- Entidades: Cliente, Pedido, Produto
- Relacionamento: Pedido contém Produto (N:M)
- Crie as tabelas com PK e FK.





Aula 22 - Modelo Lógico Relacional e Normalização

Atividade Prática 2

Identifique a **chave primária**, **estrangeira** e **candidata** no seguinte cenário:

Tabela Aluno: (id_aluno, cpf, email, nome).

Tabela Curso: (id_curso, nome).

Tabela Matricula: (id_aluno, id_curso, data).





Aula 22 - Modelo Lógico Relacional e Normalização



Atividade Prática 3

A tabela abaixo está **NÃO normalizada**. Normalize até a 3FN:

| PedidoID | Cliente | EndereçoCliente | Produto | PreçoProduto |



Aula 22 - Modelo Lógico Relacional e Normalização



Exercício em SQL

A tabela abaixo está **NÃO normalizada**. Normalize até a 3FN:

| PedidoID | Cliente | EndereçoCliente | Produto | PreçoProduto |



Aula 22 - Modelo Lógico Relacional e Normalização

Exercício em SQL

Exemplo estilo VS Code:

```
sql
-- Criando tabela normalizada
CREATE TABLE Cliente (
  id_cliente INT PRIMARY KEY,
  nome VARCHAR(100),
  endereco VARCHAR(200)
);

CREATE TABLE Pedido (
  id_pedido INT PRIMARY KEY,
  id_cliente INT,
  FOREIGN KEY (id_cliente) REFERENCES Cliente(id_cliente)
);

CREATE TABLE Produto (
  id_produto INT PRIMARY KEY,
  nome VARCHAR(100),
  preco DECIMAL(10,2)
);

CREATE TABLE PedidoProduto (
  id_pedido INT,
  id_produto INT,
  PRIMARY KEY (id_pedido, id_produto),
  FOREIGN KEY (id_pedido) REFERENCES Pedido(id_pedido),
  FOREIGN KEY (id_produto) REFERENCES Produto(id_produto)
);
```





Aula 22 - Modelo Lógico Relacional e Normalização



Modelo Relacional no Back-End

- Em Python: ORM (SQLAlchemy, Django ORM) → transforma tabelas em classes
- Exemplo SQLAlchemy:

python

```
class Aluno(Base):  
    __tablename__ = 'aluno'  
    id = Column(Integer, primary_key=True)  
    nome = Column(String)
```




Aula 22 - Modelo Lógico Relacional e Normalização

Modelo lógico relacional da loja virtual

Exemplo de tabelas:

- cliente, pedido, produto, item_pedido, categoria
- Com FKs e PKs definidos





Aula 22 - Modelo Lógico Relacional e Normalização

Regras para criar o modelo lógico

- Nome claro para tabelas e colunas
- Definir PKs e FKs desde o início
- Tabelas de junção para N:N
- Atributos normalizados (1 valor por campo)





Aula 22 - Modelo Lógico Relacional e Normalização



Resumo

- Modelo lógico relacional → traduz ER para tabelas
- Chaves: primária, estrangeira, candidata, alternativa
- Normalização até 3FN → elimina redundância e inconsistências
- Exercícios de fixação aplicados



Aula 22 - Modelo Lógico Relacional e Normalização



Biblioteca unittest (Python)

```
import unittest

class TesteSimples(unittest.TestCase):
    def test_soma(self):
        self.assertEqual(2 + 3, 5)

if __name__ == '__main__':
    unittest.main()
```



Aula 22 - Modelo Lógico Relacional e Normalização



Alguma dúvida ?

clara.gomes@ufpe.br



SOFTEX
PERNAMBUCO

 **Softex**

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO

GOVERNO FEDERAL
BRASIL
UNIÃO E RECONSTRUÇÃO